

Quiz — 2.2.2 Computational Methods — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Section A (1 mark each = 8)

Q	Ans	Why
A1	B	A computationally solvable problem can be decomposed and expressed as a clear, finite, rule-based sequence of steps.
A2	B	Decomposition = breaking a complex problem into smaller, manageable sub-problems.
A3	B	Divide and conquer splits into smaller instances of the same problem and combines results — naturally recursive.
A4	A	A heuristic gives a quick good-enough (not guaranteed optimal) answer, e.g. satnav A*.
A5	C	Returning to the last decision point after a dead end is backtracking .
A6	B	Searching large data sets for hidden patterns/correlations is data mining .
A7	B	Pipelining overlaps stages so different items are processed simultaneously (e.g. CPU fetch–decode–execute).
A8	B	Dropping irrelevant detail (exact distances) while keeping structure is representational abstraction .

Section B (12)

B1. [2] Any two (1 each): can be **decomposed** into sub-problems; has clear **inputs, processes and outputs**; is repetitive / rule-based (algorithmic); allows useful **abstractions**; data can be **structured/modelled**; has a clear **finite** sequence of steps.

B2. [3] (½ mark each, 6 correct = 3 marks; round to whole marks: 5–6 correct = 3, 3–4 = 2, 1–2 = 1) - 1 → **B** (backtracking → Sudoku dead end) - 2 → **C** (data mining → basket analysis) - 3 → **E** (heuristics → A near-optimal) - 4 → **A** (performance modelling → server load) - 5 → **D*** (visualisation → heat map)

B3. [2] - Representational abstraction (1): remove/hide detail not relevant to the problem, keeping only what is needed — e.g. the London Tube map keeps line connections but drops true distances/geography. - **Abstraction by generalisation** (1): group things by **shared characteristics** so they can be treated the same — e.g. classifying many shapes as "polygon", or grouping car types as "vehicle".

B4. [2] - 1 mark: split the network/route into smaller sub-sections (smaller instances of the same routing problem), solve each part, then **combine** the partial results into the full route. - 1 mark: it pairs with **recursion** because each smaller instance is the **same type of problem**, so the same subroutine can call itself on the sub-parts until a base case (a trivially small section) is reached.

B5. [2] - 1 mark: an exact method must check an enormous number of route combinations — it is **intractable** / grows factorially, so it is too slow for many cities. - 1 mark: a **heuristic** uses a rule of thumb to find a **good-enough** route quickly, accepting it may **not be the optimal** one.

B6. [1] Any one: patterns/trends/correlations/outliers become easier to spot than in raw figures; aids quicker decision-making; communicates findings clearly.

Section C (5)

(a) **[1] Backtracking** (accept "recursive backtracking").

(b) **[3]** Up to 3 from: - Place queens one at a time, column by column (or row by row), choosing a safe square for each (1). - After each placement, check the partial arrangement is still valid (no two queens attack) (1). - On reaching a **dead end** — no safe square in the next column — **backtrack**: undo the last placement and try the next available square for the previous queen (1). - Repeat until all 8 are placed (a solution) or all options are exhausted (1).

(c) **[1]** Any one: the search space grows very large / many combinations must be explored; lots of backtracking is needed as the board size increases (combinatorial explosion / intractable for large N).

Total: 8 + 12 + 5 = 25.